
```
boolean dfsRec(boolean[][] M, int n, int r, int c, int prev_r, int prev_c)
```

```

visited[r][c] = true
int[] d_r = {-1, 1, 0, 0}
int[] d_c = {0, 0, -1, 1}
for k = 1 to 4 do
    int new_r = r + d_r[k]
    int new_c = c + d_c[k]
    if 1 ≤ new_r ≤ n and 1 ≤ new_c ≤ n and new_r ≠ prev_r and new_c ≠ prev_c then
        if visited[new_r][new_c] then
            return true
        else if not M[new_r][new_c] then
            if dfsRec(M, n, new_r, new_c, r, c) then
                return true
return false

```

La complessità è quella di una visita in profondità che deve toccare tutti i nodi, quindi $O(n^2)$.

Note sulla correzione In molti hanno dimenticato di escludere l'arco che torna sul nodo da cui si è appena arrivato. Ripassate l'algoritmo di ricerca cicli su grafi non orientati. In assenza di quel controllo, due qualunque caselle vicine vengono considerate un ciclo e viene restituito **true** anche per l'esempio di sinistra. Sono stato buono e ho dato una penalità di -10%. Se combinata con altri pasticci, tuttavia, il voto può essere inferiore.

Esercizio B1 – Vie di fuga – Punti ≥ 8

Il problema può essere risolto tramite una rete di flusso $G = (V, E, c, s, t)$.

L'insieme dei nodi V è dato da:

- Un nodo sorgente s ;
- Un nodo pozzo t ;
- Una coppia di nodi $in_{r,c}$ e $out_{r,c}$ per ogni posizione (r, c) tale che $1 \leq r \leq R$ e $1 \leq c \leq C$. La coppia serve a garantire che attraverso un posto possa passare un solo studente.

Il numero totale di nodi è $|V| = 2RC + 2$.

L'insieme degli archi E è dato da:

- Per ogni posizione (r, c) , un arco che connette $in_{r,c}$ con $out_{r,c}$, per un totale di RC archi.
- Per ogni posizione (r, c) tale che $1 \leq r \leq R$ e $1 \leq c \leq C$, aggiungiamo gli archi
 - $(out_{r,c}, in_{r+1,c})$ (alto, se $r < R$)
 - $(out_{r,c}, in_{r-1,c})$ (basso, se $r > 1$)
 - $(out_{r,c}, in_{r,c+1})$ (destra, se $c < C$)
 - $(out_{r,c}, in_{r,c-1})$ (sinistra, se $c > 1$),

per un totale di $4RC - 2R - 2C$ archi (ogni posizione della prima e ultima riga perde un arco verso l'alto/basso, ogni posizione della prima e ultima colonna perdono un arco sinistra e destra. Non ho bisogno di trattare diversamente gli angoli, che perdono archi in due direzioni, ma sono già contanti nella prima/ultima riga/colonna).

- Per ogni posizione (r_i, c_i) corrispondente all' i -esimo studente, aggiungiamo l'arco $(s, (in_{r_i, c_i}))$, per un totale di N archi).
- Per ogni posizione del bordo, aggiungiamo gli archi verso il pozzo:
 - $(out_{r,1}, t)$ (bordo sinistro, con $1 \leq r \leq R$)
 - $(out_{r,C}, t)$ (bordo destro, con $1 \leq r \leq R$)
 - $(out_{1,c}, t)$ (bordo alto, con $1 \leq c \leq C$)
 - $(out_{R,c}, t)$ (bordo basso, con $1 \leq c \leq C$)

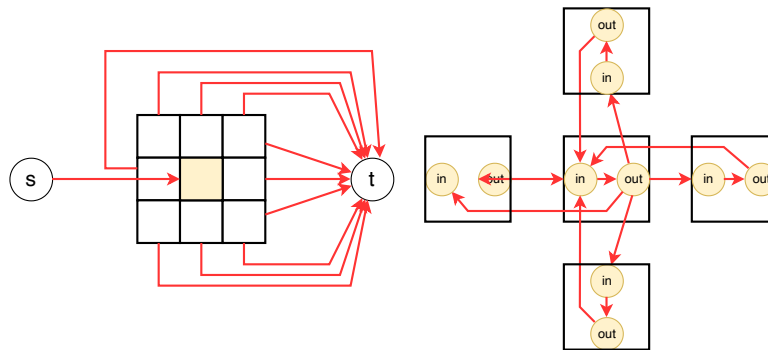
per un totale di $2R + 2C - 4$ archi (ogni posizione della prima e ultima riga, ogni posizione della prima e ultima colonna, gli angoli devono essere contati una volta sola).

Il numero totale di archi è quindi pari a $RC + 4RC - 2R - 2C + 2R + 2C - 4 + N = 5RC - 4 + N = O(RC + N)$. La funzione di capacità c è definita come:

$$c(x, y) = \begin{cases} 1 & \text{se } (x, y) \in E \\ 0 & \text{se } (x, y) \notin E \end{cases}$$

Utilizzando l'algoritmo di Ford-Fulkerson, troviamo se esistono vie di fuga disgiunte per ognuno degli studenti. Se $|f^*|$ è pari a N , abbiamo trovato le vie di fuga per tutti e restituiamo **true**, altrimenti restituiamo **false**. La complessità è pari a $O((|V| + |E|) \cdot |f^*|) = O((RC + N) \cdot |f^*|)$. Sapendo che $N \leq RC$ (al massimo uno studente per posizione) e $|f^*| \leq N$, abbiamo che la complessità finale è $O(RCN)$.

Il grafo può essere descritto come nella figura seguente. A sinistra i collegamenti riguardanti la sorgente e il pozzo, a destra i collegamenti fra una posizione e i suoi quattro vicini.



Note di correzione Solo due studenti si sono resi conti che i nodi vanno raddoppiati, per garantire che ogni posto venga attraversato una volta sola. Poiché la stragrande maggioranza ha risolto il problema senza tener conto di questo aspetto, ho deciso comunque di premiare quelle soluzioni e ho premiato in modo particolare quegli studenti con 110%.

Esercizio B2 – Hamilton – Punti ≥ 10

Essendo un problema NP-completo, utilizziamo la tecnica del backtrack per cercare tutti i possibili cicli.

Nella funzione principale, viene inizializzato un array di booleani, *visited*, per tracciare quali nodi sono stati visitati.

La base della ricorsione è data dal primo **if**: se la lunghezza del percorso è uguale al numero di nodi nel grafo e c'è un collegamento diretto dal nodo corrente al nodo di partenza, allora un ciclo Hamiltoniano è stato trovato e viene restituito **true**.

Altrimenti, si marca il nodo corrente come visitato e per ogni nodo adiacente non visitato, l'algoritmo prova a estendere il percorso. Se una chiamata ricorsiva restituisce **true**, significa che un ciclo valido è stato trovato attraverso quel percorso, e l'algoritmo può terminare anticipatamente con successo.

Se nessun percorso valido viene trovato passando attraverso un nodo specifico, il nodo viene marcato come non visitato, permettendo all'algoritmo di provare differenti percorsi.

Se tutte le opzioni sono state esplorate senza trovare un ciclo, l'algoritmo restituisce **false**.

```
boolean hamilton(GRAPH G)
```

```
    boolean[] visited = new boolean[G.n] = { false }
```

```
% Init to false
```

```
    if visitRec(G, 1, 1, visited) then
```

```
        return true
```

```
    return false
```

```
boolean visitRec(GRAPH  $G$ , int  $ln$ , NODE  $u$ , boolean[]  $visited$ )
```

```

if  $ln == G.n$  and  $1 \in G.adj(u)$  then
    return true
else
     $visited[u] = \text{true}$ 
    foreach  $v \in G.adj(u)$  do
        if not  $visited[v]$  then
            if visitRec( $G, ln + 1, v, visited$ ) then
                return true
     $visited[u] = \text{false}$ 
return false

```

La complessità dell'algoritmo è $O(n!)$, corrispondente al caso di un grafo completo.

Note di correzione Quali sono gli errori più ricorrenti che ho visto?

- Riga $visited[u] = \text{false}$ assente; in questo modo l'algoritmo fa una DFS che non passa per tutti i possibili percorsi, quindi è possibile che non trovi il percorso cercato. Il motivo dell'errore può essere una svista oppure una mancata comprensione di come si realizza il backtrack. In alcuni casi ho dato -20%, in altri sono stato più punitivo perché altre parti della descrizione sembravano indicare una mancata comprensione.
- Espressione $ln == G.n$ mancante; in questo caso, l'algoritmo chiude il ciclo prima di aver raggiunto tutti i nodi e dichiara la presenza di un ciclo che contiene il primo nodo. Anche in questo caso -20%.

Esercizio B3 – Norma L1 – Punti ≥ 12

Il problema può essere risolto tramite programmazione dinamica, utilizzando come struttura quella alla base del problema LCS (Longest Common Subsequence).

Sia $DP[i][j]$ la massima norma L1 che si può ottenere da una sottosequenza dei primi i valori di V_1 e dai primi j valori di V_2 . Tale valore può essere calcolato nel modo seguente:

- Se $i = 0$ oppure $j = 0$, non posso combinare valori di un vettore con l'altro e la norma massima è pari a 0.
- Altrimenti, se entrambi gli indici sono positivi, sono date tre possibilità che devono essere selezionate prendendo il valore massimo:
 - scarto un valore di V_1 , riducendo il problema a $DP[i - 1][j]$;
 - scarto un valore di V_2 , riducendo il problema a $DP[i][j - 1]$;
 - prendo $V_1[i]$ e $V_2[j]$, riducendo il problema a $DP[i - 1][j - 1]$ a cui sommo la differenza $|V_1[i] - V_2[j]|$.

La formula risultante è quindi

$$DP[i][j] = \begin{cases} 0 & \text{se } i = 0 \text{ or } j = 0 \\ \max\{DP[i - 1][j], DP[i][j - 1], DP[i - 1][j - 1] + |V_1[i] - V_2[j]|\} & \text{altrimenti} \end{cases}$$

La formula sopra può essere trasformata in questo algoritmo:

```
int maxL1(int[]  $V_1$ , int[]  $V_2$ , int  $n_1$ , int  $n_2$ )
```

```

int[][]  $DP = \text{new int}[0 \dots n_1][0 \dots n_2] = \{0\}$ 
for  $i = 1$  to  $n_1$  do
    for  $j = 1$  to  $n_2$  do
         $DP[i][j] = \max(DP[i - 1][j], DP[i][j - 1], DP[i - 1][j - 1] + |V_1[i] - V_2[j]|)$ 
return  $DP[n_1][n_2]$ 

```

Il costo dell'algoritmo è ovviamente $O(n_1 \cdot n_2)$.